Exercices de Compléments d'algorithmique

Série 2 – Programmation orientée objet
(26 février 2026)

Département d'Informatique – Faculté des Sciences – UMONS

Pré-requis : Notions d'orienté objet, héritage, liste chainée, pile et file (cours jusqu'au **Chapitre 3**).

Objectifs : Se familiariser avec l'héritage en Python et les listes chaînées.

---

1 La commandante Lischai de la DS Academy

Alors que vous pensiez être au bout de la DS Academy, la commandante Lischai vient rompre vos espoirs. Ce n'est que le début de l'année, forcément, vous avez encore un peu de chemin devant vous. Sans perdre de temps, elle vous cite trois mots que vous allez utiliser aujourd'hui. Vous n'avez pas souvent peur, mais ces trois mots vous effraient : « Programmation orientée objet ».

Alors que vous retenez vos cris de terreur, elle vous explique que c'est en réalité tout à fait possible de comprendre si vous vous entraînez un peu. Ce n'est pas forcément facile au début, mais ça devient non seulement facile, mais aussi intuitif avec un peu de temps. Vous sentez-vous rassuré · e pour autant ? Peut-être pas, mais comme vous avez de l'espoir et de la motivation, ça devrait bien se passer. (Pas vrai ?)

Pour vous rassurer, elle vous cite à nouveau deux mots que vous connaissez bien : « liste chainée ». Vous voilà heureux ! Vous connaissez le code, votre professeur de théorie vous l'a expliqué et donné. En tant que bon étudiant, il est évident que vous l'avez relu et compris. Vous pouvez donc reprendre le code vu en théorie sur les listes chaînées et le donjon [1].

Elle vous explique que vous allez créer un donjon « un peu à la DOOM ». Autrement dit, ce n'est plus plusieurs joueurs qui vont attaquer un monstre. C'est un seul joueur qui va attaquer (ou se faire attaquer par) beaucoup de monstres. Évidemment, il n'y aura pas qu'un seul type de monstre. Vous aurez des monstres plus forts et plus faibles.

Cependant, avant de pouvoir vous lancer dans le massacre de monstres fictifs, il faut quand même préparer un peu le terrain ! Vous commencez simplement par modifier un peu le système actuel :

1 Commencez par ajouter un attribut de résistance à un joueur. Cet attribut sera calculé comme $x * niveau$ où *niveau* représente le niveau de votre joueur, et x est un coefficient déterminé selon la classe de votre joueur. Si c'est un guerrier, $x = 1.4$. Si c'est un mage, $x = 1$.

2 Modifiez ensuite la classe `Joueur` afin qu'un joueur ne subisse que $degats - resistance$ dégâts. Par exemple, si le joueur a une résistance de 10, alors une attaque de 25 dégâts ne lui en fera en réalité que 15 ($= 25 - 10$). Notez qu'un joueur ne peut pas regagner de la vie via la résistance. Si une attaque fait 5 dégats et que votre joueur a une résistance de 15, il ne gagne (ni perd) aucun point de vie.

Parfait, ça, c'est fait. Maintenant, les monstres :

3 Créez une nouvelle classe `Monstre`. Un monstre est un joueur, mais un peu plus spécialisé (i.e., plus précis). Un monstre a une résistance faible (coefficient de résistance $x = 0.3$), et son pseudo sera toujours "Monstre". Aussi, au vu qu'il n'est pas supposé évoluer par lui-même, il ne gagne jamais d'expérience. (Imaginez qu'il vous batte et qu'il devienne encore plus difficile à battre.) Faites donc en sorte qu'un monstre ne gagne jamais d'expérience.

1. Si vous étiez distrait pendant la théorie, il est disponible sur Moodle.

4 Enfin, créez une classe `Boss`. C'est un monstre, en plus puissant. Il a une plus grande résistance (1.5), en plus de 3 barres de vie (avec une seule barre de vie, ce n'est pas drôle). Les barres de vie fonctionnent en régénérant sa vie lorsqu'une barre de vie s'épuise. S'il a 100 points de vie, et qu'il les perd, il perd une barre de vie, et l'on régénère sa vie à 100 points de vie. Il ne meurt que quand il n'a plus de barre de vie et que ses points de vie sont épuisés. Réfléchissez bien à quelle(s) méthode(s) vous devez modifier afin d'implémenter les barres de vie avant de vous lancer tête baissée dans le code.

Vous voilà prêt ! Il ne vous reste plus qu'à faire une petite simulation. La commande Lischai vous explique que la simulation se fait tour par tour. Votre joueur affrontera une première série de monstres. Il affronte les monstres un à un, et seul le monstre qu'il est en train d'affronter l'attaque en retour. (Les monstres font d'un certain point de vue la *file* pour affronter le joueur. Chaque monstre veut prouver qu'il est capable de battre le joueur seul.) Une fois la série vaincue, le joueur se soigne entièrement, récupérant intégralement ses points de vie. Nous passons ensuite au tour suivant avec une seconde série de monstres. Une fois qu'il aura tout affronté, le joueur se soignera à nouveau, et passera à la série suivante (et ainsi de suite). Ce cycle s'exécute jusqu'à ce que le joueur meurt. À chaque cycle, n'hésitez pas à afficher quelques informations (tel que le niveau du joueur, le monstre qu'il affronte actuellement, etc.).

5 Définissez maintenant une classe `Donjon`, contenant un attribut `monsters` qui représente une *file* de monstres à attaquer au prochain tour. Une méthode `generate_monsters()` génère un nombre aléatoire (dans $[n-3, n+3]$) de monstres de niveau aléatoire (entre $[n-3, n+3]$) où n est le niveau du joueur. Si le numéro de tour est un multiple de 5, la fonction génère un seul boss de niveau n. Le(s) monstre(s) généré(s) sont ajouté(s) à la file. Une méthode `fight()` permettra de simuler un tour dans le donjon. Vous pouvez supposer que c'est toujours le joueur qui attaque en premier, et ce pour chaque monstre.

La commandante Lischai vous félicite d'être arrivé · e jusqu'ici et d'avoir terminé · e cette partie.

2 La vraie section liste chaînée

La commandante Lischai remarque qu'elle vous avait promis des listes chaînées. Jusqu'ici, vous les avez beaucoup utilisées, mais pas vraiment implémentées. Afin de respecter sa parole, elle vous donne quelques exercices en plus, rien que pour vous et votre bonheur. Premièrement, insérer un élément à la fois dans une liste chaînée, c'est chouette. Insérer plusieurs éléments à la fois dans une liste chaînée, c'est mieux.

6 Implémentez une méthode `insert_all` dans la classe `ListeChainee`, permettant d'insérer tous les éléments d'une liste Python à la fin de la liste chaînée.

7 Modifiez le constructeur de votre classe `ListeChainee` afin d'ajouter un argument `elements`. Si cet argument n'est pas `None`, il représente alors une liste d'éléments à ajouter par défaut à la fin de votre liste chaînée.

C'est super, nous pouvons insérer des éléments, un ou plusieurs à la fois, à notre liste chaînée. Cependant, jusqu'ici, nous n'avons jamais prêté attention à l'ordre d'insertion de nos éléments. Parfois, il nous est utile d'imposer un ordre (croissant ou décroissant) sur la manière dont on insère dans notre liste chaînée. Afin d'exemplifier, prenons l'exemple d'un jeu bien connu : les tours de Hanoï.

Le problème des tours de Hanoï est constitué de 3 tiges (les tours) et n disques de diamètre allant de 1 à n. Au départ, les n disques sont placés selon leur diamètre sur la tige à gauche (le plus grand disque est en bas, le plus petit disque en haut). Votre but est de déplacer ces n disques vers la tige de droite. Cependant, il y a trois règles cruciales. Premièrement, vous ne pouvez intéragir qu'avec le disque en haut de chaque tour. Ensuite, vous ne pouvez déplacer qu'un seul disque à la fois, mais sur la tour que vous souhaitez. Enfin, vous ne pouvez jamais poser un disque sur un disque plus petit que lui. La Figure 2 vous montre l'état de début et de fin du problème des tours de Hanoï avec $n = 3$ disques.

Notre but sera d'implémenter le jeu des tours de Hanoï. Vous verrez plus tard [2] comment résoudre ce problème automatiquement.

2. Second TP de « Algorithmique et Programmation Orienté Objet »



FIGURE 1 – L'état de début (à gauche) et de fin (à droite) du problème des tours de Hanoï avec $n = 3$.

Vous pouvez voir les tours comme des piles. Vous ne pouvez intéragir qu'avec le premier élément de chaque tour (ce qui est le cas des piles), et vous placez toujours les nouveaux éléments tout en haut de la tour (ce qui est le cas des piles). Cependant, les piles doivent être triées afin d'imposer qu'un plus grand disque ne peut jamais se poser sur un plus petit disque. Nous devons donc faire attention à l'ordre d'insertion de nos éléments dans la pile afin de ne pas laisser l'utilisateur tricher.

8 Commencez par créer une classe `PileTriee` représentant une pile où nous devons insérer les éléments par ordre décroissant (du plus grand d'abord au plus petit, afin de respecter les règles du jeu). Notez qu'une pile triée est une pile, excepté que nous ajoutons une contrainte. Via des exceptions, faites en sorte d'imposer qu'un élément ne peut jamais se trouver au-dessus d'un élément plus petit que lui dans la pile.

9 Créez ensuite une classe `HanoiTowers` représentant un jeu des tours de Hanoï. Représentez les trois tours via des piles numérotées de 1 à 3. Si vous avez n disques, vous pouvez numéroter les n disques de 1 (le plus petit) à n (le plus grand). Cette classe contient deux méthodes :
- `moveDisk` prend le premier disque d'une tour, et l'ajoute sur une autre (les deux tours sont données en argument) ;
- `checkWin` retourne si vous avez gagné le jeu ou non.

Une variante du jeu impose que nous jouons avec les disques à l'envers. Autrement dit, la dernière règle dicte maintenant qu'un disque ne peut jamais être placé sur un disque plus grand que lui (au contraire de plus petit).

10 Ajouter un argument `reverse` à votre classe `PileTriee`, indiquant que la pile doit être triée dans l'ordre croissant (et non plus décroissant).

3 Exercices supplémentaires (héritage multiple et MRO)

La commande Lischai vous félicite d'être arrivé aussi loin. Vu que vous êtes là, on peut aller un peu plus dans les détails de l'héritage en Python, n'est-ce pas ?

Alors que la commande Lischai ouvre son syllabus avec un rire qui vous semble sadique, vous voyez inscrit « Héritage multiple » comme nom de chapitre. Bizarrement, le mot « multiple » ne vous rassure pas trop... Retenant votre fuite par curiosité, elle vous explique que Python, au contraire de certains langages (comme le Java par exemple), autorise une classe a hériter de plusieurs autres classes. Autrement dit, une même classe peut avoir plusieurs classes mère. C'est ce qu'on appelle l'*héritage multiple*. Au contraire, l'*héritage simple* correspond à un héritage où une classe ne peut hériter que d'une seule autre classe. Vous voilà rassuré, ce n'était pas si compliqué que ça en fait.

Elle continue de vous expliquer que l'héritage multiple a plusieurs cas d'utilisations et nous simplifie parfois la vie. Il faut cependant comprendre (ou avoir une intuition) sur comment il fonctionne pour que ça s'applique.

Un exemple de cas d'utilisation concret est justement notre pile triée. Imaginez avoir les classes `ListeChainee`, `Pile`, `File` et `ListeTriee` (qui correspond à une liste imposant que l'insertion des éléments se fasse de manière triée).

11 Si vous considérez avoir un héritage simple, comment feriez-vous pour implémenter les classes `PileTriee` et `FileTriee` comme introduite ci-dessus ? Prenez un instant pour réfléchir aux points positifs et négatifs avant de lire la suite de l'énoncé.

Considérez maintenant l'héritage multiple. Vous souhaitez avoir une pile triée. Vous souhaitez donc avoir les méthodes et le comportement de `Pile`, tout en y ajoutant le comportement de `ListeTriee`. Vous pouvez donc hériter des deux classes, afin « d'obtenir les propriétés des deux classes en même temps ».

Les étudiants consciencieux se demandent peut-être quelle classe mère représente alors `super()`. Comment est-ce que Python décide si `super().methode()` invoque la méthode de la première classe mère ou de la deuxième ?

La réponse simplifiée est que Python, de manière presque invisible pour vous, essaie de linéariser votre héritage. Autrement dit, il essaie de déterminer l'ordre dans lequel il cherchera vos classes mères pour vos différents attributs et méthodes. L'ordre dans lequel Python cherche est appelé « Method Resolution Order » (« ordre de résolution de méthode » si l'on traduit littéralement), ou MRO. Il est accessible via l'attribut `__mro__` (défini automatiquement pour toute classe). Dans le MRO, chaque classe a maximum un parent (de manière similaire a un héritage simple). Vous pouvez voir le MRO comme une liste de toutes les classes dont hérite une classe A, dont le premier élément dans cette liste est la classe où Python cherche une méthode en premier, le deuxième est la classe où Python cherche la méthode en deuxième, et ainsi de suite.

Par exemple, lorsque que vous appellerez `insertFirst` sur `PileTriee`, Python pourrait choisir de d'abord chercher la méthode `insertFirst` dans votre classe `PileTriee`, ensuite dans `Pile`, ensuite dans `ListeTriee` et enfin dans `ListeChainee`. Notez que Python cherchera aussi dans une classe `Object` en dernier, classe mère de toute classe, qui représente les méthodes par défaut définies automatiquement par Python. Votre MRO serait donc `[PileTriee, Pile, ListeTriee, ListeChainee, Object]`. Cet MRO s'applique aussi aux constructeurs. Si vous appelez `super()` dans `PileTriee`, vous ferez appel au constructeur de `Pile` (l'élément juste après dans le MRO).

Pour votre information, sachez que cette linéarisation n'est pas toujours possible. Un MRO n'existe pas toujours. Dans ce cas, Python lancera une erreur. La réponse plus compliquée n'est pas abordée ici. Cependant, les étudiants curieux peuvent lire la documentation officielle [3] (en anglais) afin d'en savoir plus.

12 Commencez par implémenter la classe `ListeTriee` imposant que les éléments soient insérés dans un ordre trié. Une exception est lancée pour tout essai d'insertion cassant le tri. Comme pour `PileTriee`, faites en sorte d'avoir un argument booléen `reverse` dans votre constructeur.

13 En utilisant l'héritage multiple, faites en sorte d'implémenter les classes `PileTriee` et `FileTriee` représentant respectivement une pile et une file imposant toutes deux que les éléments soient insérés d'une manière triée. Est-ce que cette implémentation a résolu les points négatifs que vous avez remarqué à la question 11 ?

Maintenant que vous savez utiliser l'héritage multiple, la commandante Lischai vous met en garde de ne pas en abuser. L'héritage multiple peut parfois introduire des difficultés de lecture, des bugs inattendus ou difficiles à comprendre lorsqu'on en abuse.

Afin de terminer votre journée en beauté, la commandante Lischai vous demande d'implémenter maintenant une file de priorité. Une file de priorité est simplement une file, où chaque élément a une priorité donnée. L'élément de plus haute priorité est celui qui est retourné en premier. Il y a plusieurs manière de la représenter. Une manière est de considérer que chaque objet inséré dans la file a une méthode `get_priority()`, retournant sa priorité sous forme d'entier. Au fil de votre cursus, vous verrez d'autres manières de faire la même chose.

14 Implémentez une file de priorité. Afin de simplifier, vous pouvez considérez que les éléments doivent être inséré dans un ordre de priorité décroissante (l'élément de plus haute priorité sera inséré d'abord, suivi par ceux de plus basse priorité). A vous de voir si vous avez besoin de l'héritage multiple ou non a cette fin.

3. `https://docs.python.org/3/howto/mro.html`